

POWER QUERY EDITOR

1. Split Column

Ways to Split a Column

1. By Delimiter
 2. By Number of Characters
 3. By Digit to Non-Digit
 4. By Lowercase to Uppercase
 5. By Positions
 6. Into Rows
-

2. Merge Queries (Joins)

Types of Joins

1. Left Outer (most common)
 2. Right Outer
 3. Full Outer
 4. Inner Join
 5. Left Anti (find records NOT matching)
 6. Right Anti
-

3. Append Queries (Stack Tables)

Two Types

1. Append as New Query
 2. Append to Existing Query
-

4. Conditional Column

Options

- If-Then-Else logic
- Nested conditions

- Comparison operators: >, <, >=, <=, =
 - Text rules: contains, begins with, ends with
-

5. Replace Values

Replace Types

1. Replace a single value
 2. Replace multiple occurrences
 3. Replace specific part of text
 4. Replace errors
-

6. Remove Rows

Options

1. Remove Top Rows
 2. Remove Bottom Rows
 3. Remove Alternate Rows
 4. Remove Blank Rows
 5. Remove Errors
 6. Remove Duplicates
-

7. Keep Rows

1. Keep Top Rows
 2. Keep Bottom Rows
 3. Keep Range of Rows
 4. Keep Duplicates
 5. Keep Errors
-

8. Filter Rows

Types

- Number filters
 - Text filters
 - Date filters
 - Custom filters
 - Basic vs Advanced filtering
-

9. Group By

Modes

1. **Basic**
 - Single column
2. **Advanced**
 - Multiple columns

Aggregations

- Sum
 - Count rows
 - Min / Max
 - Average
 - Median
 - Count distinct
 - All rows (to create sub-tables)
-

10. Add Column

Common Add Column Options

1. Custom Column (M-code formula)
2. Index Column (0-based / 1-based / custom)
3. Conditional Column
4. Invoke Custom Function
5. Add Column From Examples

11. Column From Examples

Uses

- Extract first name
- Extract domain from email
- Format phone numbers
- Auto detect patterns

12. Data Type Conversion

Types

- Whole Number
- Decimal Number
- Text
- Date
- Date/Time
- Boolean

13. Pivot & Unpivot

Pivot Columns

- Convert rows → columns

Unpivot Columns

- Convert columns → rows

14. Formatting Columns

Text Formatting

- Trim
- Clean (remove non-printable characters)
- UPPER

- lower
- Proper Case

Number Formatting

- Round
 - Round Up
 - Round Down
 - Divide / Add new calculated numeric fields
-

15. Date Functions

Create new date columns

- Year
- Month
- Month Name
- Quarter
- Week Number
- Day Name

Duration Functions

- Date difference
 - Age
 - Duration in days/hours/minutes
-

16. Parameter Creation

Parameter Types

- Text
- Number
- List of Values
- Query-based list
- Current Value

17. Manage Queries

Options

- Duplicate
- Reference
- Rename
- Organize into folders (groups)

18. Query Dependencies

Shows

- Data flow
- Parent/child queries
- Staging → Transformation → Final output

19. Advanced Editor (M-Code)

Skills

- Understanding M language structure
- Editing steps
- Writing custom logic
- Combining transformations

20. Error Handling

Tools

- Remove Errors
- Replace Errors
- Column Quality (Valid/Error/Empty)

- Column Distribution
 - Column Profile
-

TOPIC: SPLIT COLUMN

Power Query provides **6 major Split Column methods**:

1. **By Delimiter**
2. **By Number of Characters**
3. **By Positions**
4. **By Lowercase to Uppercase (Split by character transition)**
5. **By Digit to Non-Digit / Non-Digit to Digit**
6. **By Repeating Delimiter**

Below is the full breakdown with examples.

Sample Data for SPLIT Column Topic

Use this dataset:

Customer Full Name	Product Code	OrderID	Comments
John Carter	FR-123-A	101	Delivered-On-Time
Maria Smith	EL-550-B	102	Return-Requested
Rahul Kumar	CL-700	103	HighValueCustomer
Priya	FN-200-C	104	NewCustomer2024

1. SPLIT COLUMN BY DELIMITER

Used when values are separated by symbols: " - ", "/", ":", "@", etc.

Business Problem:

We want Product Code split into **Category, Item No, Variant**.

Sample Value:

FR-123-A

Steps:

Home → Split Column → By Delimiter → Choose “-”

Output:

Product Code	Category	Item No	Variant
FR-123-A	FR	123	A

Business Use:

- Extracting First Name / Last Name
 - Extracting City from Address
 - Extracting Domain from Email
 - Splitting SKU codes
-

2. SPLIT COLUMN BY NUMBER OF CHARACTERS

Used when you want to split based on fixed length.

Business Problem:

Separate First 2 characters of Product Code to identify product family.

Example Value:

FR-123-A

Split after 2 characters → Result:

| FR | -123-A |

Use Cases:

- Extract country codes (first 2 letters)
 - Extract branch code (first 3 digits)
 - Extract year from “20240105”
-

3. SPLIT COLUMN BY POSITIONS

Used when you know exact character positions where splitting must occur.

Business Problem:

Split Comments column into:

- Word 1 (Delivered)
- Word 2 (On)
- Word 3 (Time)

Example Value:

Delivered-On-Time

Positions

- After index 9 (Delivered | -On-Time)
- After index 12 (Delivered | On | Time)

Output:

| Delivered | On | Time |

Use Cases:

- Splitting address like “BLR560001”
- Splitting bank account number
- Uniform length code fields

4. SPLIT BY LOWERCASE TO UPPERCASE

Power Query automatically splits when:

a lowercase letter is followed by an uppercase letter

Business Problem:

Split customer tags into readable words.

Example Value:

HighValueCustomer

Output:

| High | Value | Customer |

Business Use:

- Splitting CamelCase words
- Splitting system-generated labels

- Converting "OrderDeliveredOnTime" → readable format
-

5. SPLIT BY DIGIT TO NON-DIGIT / NON-DIGIT TO DIGIT

Very important for SKU codes, serial numbers, invoice numbers.

Example Value:

NewCustomer2024

Split Digit → Non-digit gives:

| NewCustomer | 2024 |

Example 2:

123ABC789

Split Digit to Non-digit:

| 123 | ABC789 |

Business Use Cases:

- Separate numeric part from text part
 - Extract year from strings
 - Separate item codes like "FR1200A"
 - Use in predictive analytics for code classification
-

6. SPLIT BY REPEATING DELIMITER

If delimiter appears multiple times and you want Power Query to treat them as one.

Example Value:

John--Carter

Delimiter → "-" (repeats twice)

Output:

| John | Carter |

Business Use:

- When messy data has repeated spaces/hyphens
- When software exports produce double delimiters

- Useful in PDF-extracted data lines

BONUS: SPLIT COLUMN INTO ROWS

(Students love this feature)

Example Value:

Delivered-On-Time

Split into rows:

Value

Delivered
On
Time

Used for:

- Exploding multi-value fields
 - Tag/keyword extraction
 - Converting comma-separated values to rows
-

MERGE QUERIES

Table 1: Orders (Left Table)

OrderID	ProductID	Quantity
101	P001	2
102	P002	1
103	P003	5
104	P004	3
105	P006	4

Table 2: Product Master (Right Table)

ProductID	ProductName	Price
P001	Laptop	55000
P002	Mouse	500
P003	Keyboard	1200
P005	Monitor	8000

You will Merge using the common key:

Join Column = ProductID

Explain Every Join Type with Results

1. LEFT OUTER JOIN

“All rows from LEFT table + matched rows from RIGHT”

Used for:

- Common lookup
- VLOOKUP equivalent
- Keep all orders even if the product is missing

Result Table

OrderID	ProductID	Quantity	ProductName	Price
101	P001	2	Laptop	55000
102	P002	1	Mouse	500
103	P003	5	Keyboard	1200
104	P004	3	NULL	NULL
105	P006	4	NULL	NULL

Explanation

- P001, P002, P003 matched → full data
 - P004, P006 don't exist in Product Master → right side becomes NULL
 - No rows removed
-

2. RIGHT OUTER JOIN

“All rows from RIGHT table + matched rows from LEFT”

Used for:

- You want to keep all products
- Even if there are no orders

Result Table

ProductID	ProductName	Price	OrderID	Quantity
P001	Laptop	55000	101	2
P002	Mouse	500	102	1
P003	Keyboard	1200	103	5
P005	Monitor	8000	NULL	NULL

Explanation

- P005 exists only in Product Master → included
 - No change for matched items
-

3. INNER JOIN

“Only matching rows from BOTH tables”

Used for:

- Clean data
- Remove unmatched records
- Create validated dataset

Result Table

OrderID	ProductID	Quantity	ProductName	Price
101	P001	2	Laptop	55000
102	P002	1	Mouse	500
103	P003	5	Keyboard	1200

Explanation

- Only P001, P002, P003 matched both tables
 - P004, P006, P005 removed
-

4. FULL OUTER JOIN

“All rows from both tables — matched + unmatched”

Used for:

- Full comparison
- Audit / validation
- Finding which records match & don't match

Result Table

ProductID	From Orders	From Product Master
P001	Found	Found
P002	Found	Found
P003	Found	Found
P004	Found	Not Found
P006	Found	Not Found
P005	Not Found	Found

Explanation

Everything comes, matched + mismatched.

5. LEFT ANTI JOIN

“Rows from LEFT that do NOT match RIGHT”

Helps in finding Missing Products in Master Table

Used for:

- Data quality checks
- Identify missing master data

Result Table

ProductID	OrderID	Quantity
P004	104	3
P006	105	4

Explanation

These ProductIDs are in Orders but NOT in Product Master.

6. RIGHT ANTI JOIN

“Rows from RIGHT that do NOT match LEFT”

Helps in finding Unused products

Used for:

- **Identify products that were never ordered**

Result Table

ProductID	ProductName	Price
P005	Monitor	8000

Explanation

P005 exists in Product Master but no orders.

APPEND QUERIES (STACK TABLES)

Append = Stacking rows vertically
(Equivalent to SQL UNION ALL)

Sample Datasets for Append Topic

Table: Sales_Jan

OrderID	Region	Sales
101	South	1500
102	South	2200
103	South	1800

Table: Sales_Feb

OrderID	Region	Sales
201	South	1900
202	South	2100
203	South	2500

Table: Sales_Mar

OrderID	Region	Sales
301	South	2300
302	South	1700
303	South	2600

These 3 tables represent monthly sales.

We will append them to get Quarter 1 Sales.

WHY APPEND IS USED?

- Combine monthly data into one table
 - Combine same-structure regional files
 - Combine multiple Excel sheets
 - Stack tables with same columns
 - Generate consolidated datasets
-

TYPES OF APPEND

1. APPEND AS NEW QUERY

Creates a new table containing combined data.

When to use:

- ✓ You want to keep original tables as-is
 - ✓ You want a separate consolidated table
 - ✓ Good for: Monthly / Yearly consolidation
-

Result After Appending Jan + Feb + Mar

OrderID	Region	Sales
101	South	1500
102	South	2200
103	South	1800
201	South	1900
202	South	2100

203	South	2500
301	South	2300
302	South	1700
303	South	2600

Explanation

- Rows added vertically
- No columns added
- Data becomes one continuous table

2. APPEND TO EXISTING QUERY

Adds more tables inside an existing table.

When to use:

- ✓ You want to modify an existing table
- ✓ You don't need the original files separate
- ✓ Useful for: Auto-refresh pipelines

MODES OF APPEND

A. Append Two Tables

Example

Append Sales_Jan + Sales_Feb

Result:

OrderID	Region	Sales
101	South	1500
102	South	2200

103	South	1800
201	South	1900
202	South	2100
203	South	2500

When this mode is used:

- Simple two-month data
- Combining two sheets (North + South region)
- Quick merge of two CSV files

B. Append Three or More Tables

Example

Append:

Sales_Jan + Sales_Feb + Sales_Mar

Power Query allows selecting multiple tables in one step.

Output:

OrderID	Region	Sales
101	South	1500
102	South	2200
103	South	1800
201	South	1900
202	South	2100
203	South	2500
301	South	2300

302	South	1700
303	South	2600

TOPIC: CONDITIONAL COLUMN

Power Query's Conditional Column is used to create new columns based on logic:

- If-Then-Else logic
- Multiple/Nested conditions
- Numeric comparisons
- Text-based rules
- Category creation

This is one of the TOP 3 most used features in Power Query.

Sample Dataset

EmpID	EmployeeName	Age	Sales	CustomerFeedback
101	John	22	45000	Excellent
102	Maria	34	75000	Good
103	Rahul	29	30000	Average
104	Priya	41	90000	Poor
105	David	55	120000	Excellent

We will use this table to create multiple conditional columns.

1. IF-THEN-ELSE BASIC LOGIC

Business Requirement:

Classify employees into “Young”, “Middle”, “Senior” based on Age.

Rule:

- If Age < 30 → “Young”
- If Age between 30 and 45 → “Middle”

- Else → “Senior”

Steps:

Power Query → Add Column → Conditional Column

Output Column: Age Group

EmployeeName	Age	Age Group
John	22	Young
Maria	34	Middle
Rahul	29	Young
Priya	41	Middle
David	55	Senior

2. NESTED CONDITIONS

When you need multiple layers of logic in one conditional column.

Business Problem:

Create Performance Rating using both Sales and Feedback.

Rules:

- If Sales > 100000 AND Feedback = Excellent → “Outstanding”
- Else If Sales > 70000 → “Excellent”
- Else If Sales > 40000 → “Good”
- Else → “Needs Improvement”

This is a NESTED IF-THEN-ELSE structure.

Output Column: Performance Rating

EmployeeName	Sales	Feedback	Performance Rating
John	45000	Excellent	Good
Maria	75000	Good	Excellent
Rahul	30000	Average	Needs Improvement
Priya	90000	Poor	Excellent
David	120000	Excellent	Outstanding

3. COMPARISON OPERATORS USED IN CONDITIONAL COLUMNS

You can create conditions using:

Operator	Meaning
>	Greater than
<	Less than
>=	Greater or equal
<=	Less or equal
=	Equals
<>	Not equal

Example Scenario:

Create Bonus Eligibility

Rule:

If Sales >= 80000 → “Eligible” else “Not Eligible”

Output: Bonus Eligibility

Name	Sales	Bonus Status
John	45000	Not Eligible
Maria	75000	Not Eligible
Rahul	30000	Not Eligible
Priya	90000	Eligible
David	120000	Eligible

4. TEXT RULES (Contains / Begins With / Ends With)

These are powerful when working with text columns.

Available Text Operators:

- contains
 - does not contain
 - begins with
 - ends with
-

A. Example: Create Customer Sentiment Category

Business Rules:

- If Feedback contains “Excellent” → “Promoter”
 - If Feedback contains “Good” → “Happy”
 - If Feedback contains “Average” → “Neutral”
 - If Feedback contains “Poor” → “Unhappy”
 - Else → “Unknown”
-

Output: Sentiment Category

Feedback	Sentiment
Excellent	Promoter
Good	Happy
Average	Neutral
Poor	Unhappy
Excellent	Promoter

B. Example: Create Region Category using “begins with”

Sample Data

Location:

- "SOU-CHN-001"
- "NOR-DEL-002"
- "EAS-KOL-005"

Rule:

- If Location begins with “SOU” → “South”
 - If begins with “NOR” → “North”
 - If begins with “EAS” → “East”
-

5. BUSINESS EXAMPLE: CATEGORY BASED ON SALES

Task:

Create a new column “Sales Category”

Rules:

- Sales < 40000 → “Low”
- Sales between 40000–80000 → “Medium”
- Sales > 80000 → “High”

Output: Sales Category

Name	Sales	Sales Category
John	45000	Medium
Maria	75000	Medium
Rahul	30000	Low
Priya	90000	High
David	120000	High

REPLACE VALUES

Replace Values is used to clean and standardize data by replacing:

- specific values
- partial text inside values
- multiple occurrences
- NULL values
- errors
- inconsistent spellings

This is one of the most used cleaning tools in Power Query.

Sample Dataset for Replace Values Topic

EmpID	City	Status	Rating	Comments
101	Chennai	Active	4	N/A
102	Mumbai	Inactive	Error	Excllent
103	null	Active	5	Average
104	Banglore	Active	3	Poor
105	Chennai	Active	4	NA
106	Hyderbad	Inactive	2	Excellent

This data contains:

- NULL values
- Spelling mistakes
- N/A variations
- City spelling errors
- Unwanted text values
- Error text

We will clean everything using Replace Values.

1. REPLACE A SINGLE VALUE

Business Need:

Replace inconsistent values to maintain data quality.

Example:

Replace "Banglore" → "Bangalore"

Output:

City Before	City After
Banglore	Bangalore

Use Cases:

- Fix product names
- Fix misspelled city names
- Replace old brand names

2. REPLACE MULTIPLE OCCURRENCES (Same Value Appears Many Times)

Dataset Has:

- "N/A"
- "NA"
- "N-A"
- "N/A "

Goal: Convert all these to blank.

Replace Values Steps:

Replace "N/A" → ""

Replace "NA" → ""

Output:

Comments Before	Comments After
N/A	(blank)
NA	(blank)

Use Cases:

- Cleaning missing data
- Standardizing multiple variations of NULL
- Removing placeholder text

3. REPLACE SPECIFIC PART OF TEXT

Power Query also replaces substring inside text.

Example:

Replace "Active" → "Active"

It fixes embedded text errors.

Before → After:

Status Before	Status After
Active	Active

More Examples:

- Replace "Hyderbad" → "Hyderabad"
- Replace "Excllent" → "Excellent"
- Replace "Chennai City" → "Chennai"

Business Scenarios:

- Fixing typos inside product descriptions
- Removing version numbers
- Standardizing “Ltd.” vs “Limited”

4. REPLACE NULL VALUES

NULL means *missing*.

Example:

Replace NULL in “City” → “Unknown”

Output:

City Before	City After
null	Unknown

Why Important?

- **Dashboards break if NULL exists**
 - **Replacement makes visualizations cleaner**
 - **Helps grouping and segmentation**
-

5. REPLACE ERRORS

Errors appear due to:

- **Wrong data type**
- **Division by zero**
- **Text in numeric column**
- **Conversion errors**

Replace Errors is used to fix ALL error rows.

Example:

Rating column contains "Error"

Step:

Replace Errors → Replace with 0

Output:

Rating Before	Rating After
Error	0
4	4

5	5
---	---

Business Uses:

- Replace data conversion errors
- Avoid breaking dashboards
- Maintain numeric columns without errors

6. REPLACE TYPES (Changing Data Types Using Replace Rule)

Although not common, Replace Types means → when converting data types, Power Query may auto-replace invalid values.

Example:

Trying to convert "Error" to Number → becomes Error Row,
then using Replace Errors → replace with 0.

Example Flow:

1. Change Type → Error
2. Replace Errors → 0

EXAMPLES USING SAMPLE DATA

Below is the step-by-step cleaned output.

Example 1: NULL → “Unknown”

City Before	City After
null	Unknown

Example 2: “N/A” → blank

Comments Before Comments After

N/A	(blank)
-----	---------

NA	(blank)
----	---------

Example 3: Spelling Fix

Before After

Banglore	Bangalore
Hyderbad	Hyderabad
Excllent	Excellent

Example 4: Remove text errors

Status Before	Status After
Active	Active

Example 5: Replace Errors

If Rating = “Error”, replace with 0

Before	After
Error	0

REMOVE ROWS

Remove Rows is used for cleaning datasets by deleting unwanted rows such as:

- extra headers
- blank rows
- error rows
- duplicates
- alternate pattern rows

This is one of the first steps in data cleaning.

Sample Dataset

Row	EmpID	Name	Salary
1	EmpID	Name	Salary
2	101	John	45000
3			
4	102	Maria	55000
5	ERROR	Rahul	60000
6	103	Priya	50000
7	103	Priya	50000
8	104	David	70000
9			
10	105	John	45000

This dataset contains:

- ✓ repeated header
- ✓ blanks
- ✓ errors
- ✓ duplicates
- ✓ dirty rows

We will use **Remove Rows** to clean everything.

1. REMOVE TOP ROWS

Used when files contain:

- ✓ extra headers
- ✓ notes
- ✓ metadata
- ✓ unwanted intro rows

Example:

Remove first **1** row (because row 1 contains repeated header).

Before:

Row	EmpID	Name	Salary
1	EmpID	Name	Salary
2	101	John	45000

After:

Row	EmpID	Name	Salary
2	101	John	45000

2. REMOVE BOTTOM ROWS

Used when:

- ✓ file ends with footnotes

- ✓ extra total lines
- ✓ blank rows at bottom
- ✓ unnecessary summaries

Example:

Remove last **1** row (blank row 9 or footer row).

Before:

Row	EmpID	Name	Salary
9	(blank)	(blank)	(blank)

After:

Blank row at bottom removed.

3. REMOVE ALTERNATE ROWS

Used for:

- ✓ Sample data selection
- ✓ Removing every 2nd row from messy sensor data
- ✓ Removing noise from log files

Example Scenario:

Remove every alternate row starting from row 2.

Definition:

- Start removing at row 2
- Remove 1 row
- Keep 1 row
(Repeat)

Before:

Row	EmpID	Name
1	101	John

2	102	Maria
3	103	Priya
4	104	David
5	105	John

After:

Row	EmpID	Name
1	101	John
3	103	Priya
5	105	John

4. REMOVE BLANK ROWS

Blank rows break dashboards and cause errors.

Example:

Before:

Row	EmpID	Name	Salary
3			

After removing blank rows:

Row 3 removed.

Use Cases:

- Cleaning Excel-based datasets
 - Removing empty lines from CSV
 - Preparing consistent data for modeling
-

5. REMOVE ERRORS

Removes rows that display **Error** due to:

- invalid data type
- division by zero
- text in numeric columns
- failed transformations

Example:

Before:

EmpID	Name	Salary
ERROR	Rahul	60000

After Remove Errors:

Row removed.

Use Cases:

- Ensure numeric columns contain only numbers
 - Fix issues during refresh
 - Remove corrupt values
-

6. REMOVE DUPLICATES

Duplicates cause wrong totals, wrong counts, and wrong KPIs.

Example:

Before:

EmpID	Name	Salary
103	Priya	50000
103	Priya	50000

After:

Only 1 row kept.

Use Cases:

- Customer master
- Employee master
- Sales transaction cleanup
- Attendance logs

FULL CLEANING PROCESS (BUSINESS LEVEL)

Using all Remove Row operations:

Input Messy File:

Row	EmpID	Name	Salary
1	EmpID	Name	Salary
2	101	John	45000
3	(blank)	(blank)	(blank)
4	102	Maria	55000
5	ERROR	Rahul	60000
6	103	Priya	50000
7	103	Priya	50000
8	104	David	70000
9	(blank)	(blank)	(blank)
10	105	John	45000

Step-by-step Cleaning:

Remove Top Rows (remove repeated header)

Remove Blank Rows

Remove Errors

Remove Duplicates

Final Clean Output:

EmpID	Name	Salary
101	John	45000
102	Maria	55000
103	Priya	50000
104	David	70000
105	John	45000

KEEP ROWS in Power Query

Keeping rows means selecting only the rows you WANT to keep and removing all others.

Power Query provides 5 options:

- 1. **Keep Top Rows**
- 2. **Keep Bottom Rows**
- 3. **Keep Range of Rows**
- 4. **Keep Duplicates**
- 5. **Keep Errors**

Sample Dataset (Sales Transactions)

Use this dataset for all examples:

TransactionID	Region	Amount	Status
1001	North	5000	Completed
1002	South	4200	Completed
1003	East	3200	Failed
1004	North	6000	Completed
1005	West	ERROR	Error
1006	South	2500	Completed
1007	East	1800	Completed
1008	West	3000	Completed
1009	North	1500	Failed

1. Keep Top Rows

Keeps ONLY the first N rows.

Use Case:

Your manager asks:

“I need the first 3 transactions for testing.”

Steps:

Home → **Keep Rows** → Keep Top Rows → Enter 3

Example Output:

TransactionID	Region	Amount	Status
1001	North	5000	Completed
1002	South	4200	Completed
1003	East	3200	Failed

2. Keep Bottom Rows

Keeps ONLY the last N rows.

Use Case:

QA team needs:

“Give me the last 2 transactions to verify data entry.”

Steps:

Home → **Keep Rows** → Keep Bottom Rows → Enter 2

Example Output:

TransactionID	Region	Amount	Status
1008	West	3000	Completed
1009	North	1500	Failed

3. Keep Range of Rows

Keeps rows starting from a position and number of rows.

Format:

- **Start Index** (0-based indexing)
- **Number of rows to keep**

Use Case:

Manager says:

“Bring only rows from 3rd to 6th for middle-section analysis.”

Rows needed = Start at row 2 (because index starts from 0) and keep 4 rows.

Steps:

Home → **Keep Rows** → Keep Range of Rows

Start = **2**

Count = **4**

Output:

TransactionID	Region	Amount	Status
1003	East	3200	Failed
1004	North	6000	Completed

1005	West	ERROR	Error
1006	South	2500	Completed

4. Keep Duplicates

Keeps only rows where duplicates exist.

Modify dataset to explain duplicates:

TransactionID	Region	Amount	Status
1001	North	5000	Completed
1002	South	4200	Completed
1003	East	3200	Completed
1003	East	3200	Completed
1004	North	6000	Failed

Here, **TransactionID 1003** is duplicated.

Use Case:

Finance team:

“Show me only duplicate transactions for audit.”

Steps:

Home → Keep Rows → **Keep Duplicates**

Output:

TransactionID	Region	Amount	Status
1003	East	3200	Completed
1003	East	3200	Completed

5. Keep Errors

Keeps ONLY rows that contain errors.

Example Error Row:

TransactionID	Region	Amount	Status
1005	West	ERROR	Error

Use Case:

Your team wants to:

“Extract only rows with data issues so we can fix them separately.”

Steps:

Home → Keep Rows → **Keep Errors**

Output:

TransactionID	Region	Amount	Status
1005	West	ERROR	Error

FILTER ROWS in Power Query

Filtering rows means selecting only the rows you **want to keep based on a condition**.

Power Query provides 5 types of filters:

- 1. **Number Filters**
- 2. **Text Filters**
- 3. **Date Filters**
- 4. **Custom Filters**
- 5. **Basic vs Advanced Filtering**

Sample Dataset (Sales Orders)

OrderID	Region	Order Date	Amount	Status
101	North	2024-01-05	5000	Completed
102	South	2024-01-12	2500	Pending
103	East	2024-02-10	8000	Completed
104	East	2023-12-28	1000	Cancelled
105	West	2024-03-02	1500	Completed
106	South	2024-03-15	9500	Completed
107	West	2024-01-18	4000	Pending

1. Number Filters

Used for columns with numeric values (Amount column).

Available Options

- Equals (=)
- Not equals (≠)

- Greater Than
 - Less Than
 - Between
 - Top 10% / Top 10 Values
 - Bottom 10% / Bottom 10 Values
-

Business Question:

“Show me all orders greater than ₹5,000.”

Steps

Column **Amount** → Filter → **Number Filters** → **Greater Than** → 5000

Result:

OrderID	Region	Amount	Status
103	East	8000	Completed
106	South	9500	Completed

Another Example (Between):

“Show orders between ₹2,000 and ₹5,000.”

Amount → Number Filters → **Between**

2. Text Filters

Used for columns containing text (Region, Status).

Available Options

- Equals
- Does Not Equal
- Begins With
- Ends With
- Contains

- Does Not Contain
-

Business Question:

“Give me only East region orders.”

Steps

Column **Region** → Filter → **Text Filters** → **Equals** → **East**

Result:

OrderID	Region	Amount	Status
103	East	8000	Completed
104	East	1000	Cancelled

Another Example (Contains):

“Show orders where Status contains ‘Pend’ (Pending).”

Status → Text Filters → Contains → Pend

3. Date Filters

Used for date columns (Order Date).

Available Options

- Before
- After
- Between
- This Year / Month / Quarter
- Last Year
- Next Month
- Year-to-Date

- Custom date selection

Business Question:

“Show only orders from January 2024.”

Steps

Order Date → Filter → **Date Filters** → **In This Month** → **January 2024**

Result:

OrderID	Order Date	Amount
101	2024-01-05	5000
102	2024-01-12	2500
107	2024-01-18	4000

Another Example (Between):

“Show orders between Feb 1, 2024 and Mar 31, 2024.”

Date Filters → **Between**

4. Custom Filters

Used to apply **multiple conditions** using AND/OR.

Business Question:

“Show orders where Amount > 5,000 AND Region = South.”

Steps

Amount → Number Filters → Custom Filter

Condition 1: Amount > 5000

AND

Condition 2: Region equals South

Result:

OrderID	Region	Amount
106	South	9500

Example with OR

“Show orders from North **or** West region.”

5. Basic vs Advanced Filtering

Basic Filter (Checkbox List)

Shows all values in a column.

Good for simple filtering.

Example:

Tick only → **Completed**

Advanced Filter

Used in **Number filters**, **Text filters**, **Date filters**, **Custom filters**.

Allows **multiple conditions**, comparison, contains, begins with, etc.

Example:

Amount > 5000 AND Status = Completed

GROUP BY

GROUP BY in Power Query

Group By is used to **summarize data** (aggregation), similar to **SQL GROUP BY** or **Excel Pivot Table**.

There are two modes:

1. Basic Mode

- Group by **one** column
- Apply **one** aggregation

2. Advanced Mode

- Group by **multiple** columns
- Apply **multiple** aggregations

Sample Dataset (Sales Transactions)

OrderID	Category	Region	Amount	Units
101	Electronics	North	5000	3
102	Furniture	South	2500	1
103	Electronics	East	8000	5
104	Electronics	North	3000	2
105	Furniture	West	1500	1
106	Furniture	South	9500	4

Basic mode

Basic mode allows:

- **One group-by column**
 - **One aggregation**
-

Example 1: Total Sales by Category

Business Question:

“Give me total sales amount of each category.”

Steps

Home → Group By → **Basic Mode**

- Group By: **Category**
 - New column name: **Total Sales**
 - Operation: **Sum**
 - Column: **Amount**
-

Output

Category	Total Sales
Electronics	16000
Furniture	13500

Example 2: Count Orders by Region

Business Question:

“How many orders came from each region?”

Steps

Group By → Basic

- Group By: **Region**
- New column: **Order Count**

- Operation: **Count Rows**
-

Output

Region	Order Count
North	2
South	2
East	1
West	1

2. GROUP BY – ADVANCED MODE

Advanced mode allows:

- **Multiple grouped columns**
 - **Multiple summary calculations**
-

Example: Sales Summary by Category + Region

Business Question:

“Show total sales and average units for each category in each region.”

Steps

Group By → **Advanced Mode**

Group By Columns:

- Category
- Region

Aggregations:

New Column	Operation	Column
------------	-----------	--------

Total Sales	Sum	Amount
Avg Units	Average	Units

Output

Category	Region	Total Sales	Avg Units
Electronics	North	8000	2.5
Electronics	East	8000	5
Furniture	South	12000	2.5
Furniture	West	1500	1

Supported Aggregation Types – Explained with Examples

Below are all aggregation types with business use cases.

Sum

Use Case: Total sales per category

Operation: Sum → Amount

Count Rows

Use Case: Number of employees in each department

Operation: Count Rows

Min / Max

Use Case: Find:

- Minimum order amount
- Maximum units ordered

Average

Use Case: Average units sold per region

Median

Use Case: Median salary per department

Count Distinct Values

Use Case:

“How many UNIQUE customers per region?”

All Rows (Sub-Table)

Creates a **nested table** containing all rows of each group.

Use Case:

“You want a group-wise drilldown table.”

Example:

Group by Category → operation = **All Rows**

Output:

Each row contains a sub-table of records for that category.

Additional Real-Time Business Examples

Example 1: Sales by Category

Group By → Category

Aggregation → Sum → Amount

Example 2: Total Employees by Department

Use this HR Dataset:

EmpID	Department	Salary

E01	HR	30000
E02	IT	50000
E03	HR	32000
E04	Sales	45000
E05	IT	55000

Group By → Basic

Group By: Department

New Column: Employee Count

Operation: Count Rows

Output:

Department	Employee Count
HR	2
IT	2
Sales	1

Example 3: Min/Max Sales by Region

Region	Min Sales	Max Sales
North	3000	5000
South	2500	9500
East	8000	8000
West	1500	1500

ADD COLUMN in Power Query

The **Add Column** tab is used to create new fields based on formulas, logic, or transformations.

Common Add Column Options:

1. **Custom Column (M-code formula)**
2. **Index Column (0-based / 1-based / Custom)**
3. **Conditional Column**
4. **Invoke Custom Function**
5. **Add Column From Examples**

Sample Dataset (Sales Orders)

OrderID	Customer	Region	Amount	Order Date
101	John	North	5000	2024-01-05
102	Meera	South	2500	2024-01-12
103	Karan	East	8000	2024-02-10
104	John	North	3000	2024-02-18
105	Priya	West	1500	2024-03-02

1. Custom Column (M-Code Formula)

Used when you want to create a new column using mathematical or text formulas.

Business Requirement:

“Create a column that calculates 10% discount on Amount.”

Steps

Add Column → **Custom Column**

Formula:

=[Amount] * 0.10

Output Column: **Discount**

Result:

Amount	Discount
5000	500
2500	250
8000	800
3000	300
1500	150

Other Useful M-Code Formulas

Requirement	Formula
Full Name	[First] & " " & [Last]
Add tax (18%)	[Amount] * 0.18
Convert to text	Text.From([Amount])
Year	Date.Year([Order Date])
Month	Date.Month([Order Date])
If logic	if [Amount] > 5000 then "High" else "Low"

2. Index Column (Row Numbers)

Automatically generates sequence numbers.

Index Types:

- **0-based** → starts from 0
 - **1-based** → starts from 1
 - **Custom** → any number
-

Example

Business Requirement:

“Add a serial number starting from 1.”

Steps:

Add Column → **Index Column** → **From 1**

Output:

OrderID	Index
101	1
102	2
103	3
104	4
105	5

Custom Index Example

Start index at 500:

Value = 500 → Increment = 1

Creates: 500, 501, 502...

3. Conditional Column

(You already learned this earlier, but here it's summarized under Add Column)

Used to create logic-based columns.

Example:

Business Requirement:

“Label orders as HIGH or LOW based on Amount.”

Steps:

Add Column → **Conditional Column**

Rule:

If Amount > 5000 → High

Else → Low

Output:

Amount	Category
5000	Low
2500	Low
8000	High
3000	Low
1500	Low

4. Invoke Custom Function

Used when you:

- ✓ Create a function in Power Query
 - ✓ Want to apply it repeatedly
-

Example Scenario (Common in Real Projects)

You have multiple files monthly:

- Sales_Jan.csv
- Sales_Feb.csv
- Sales_Mar.csv

You create a function:

fxCleanSales

This function performs:

- Remove errors
- Trim text
- Fix date formats

Then:

Add Column → **Invoke Custom Function**

Select your function → apply to each file.

Business Use Case:

“We need to clean each imported Excel file using a cleaning function.”

Power Query applies the function to each row automatically.

5. Add Column From Examples

This is one of the most powerful features.

Power Query looks at **your examples** → creates formula automatically.

Example 1: Extract First Name

Customer column contains:

"Meera S"

Steps:

Add Column → **Column From Examples** → **From Selection**

Type: "Meera"

Power Query automatically identifies:

Text.BeforeDelimiter([Customer], " ")

Example 2: Create Month Name Column

Order Date = 2024-01-05

You type → "January"

Power Query generates:

`Date.MonthName([Order Date])`

COLUMN FROM EXAMPLES

Column From Examples allows you to **type the output you want**, and Power Query automatically generates the formula (M-code).

It is used when you **don't know the formula** or want **quick transformations**.

You can use it to:

- ✓ Extract first name
- ✓ Extract domain from email
- ✓ Format phone numbers
- ✓ Auto-detect repeated patterns

Sample Dataset (Employee Information)

EmpID	Full Name	Email	Phone
E101	John Mathews	john.m@abc.com	9876543210
E102	Priya Suresh	priya.suresh@xyz.in	9567812345
E103	Rohan Kumar	rohan.kumar@mega.co.in	9123456789

1. Extract First Name

Business Requirement:

“Create a new column that contains only the first name.”

Steps:

1. Select **Full Name**
2. Add Column → **Column From Examples** → **From Selection**
3. In the new column, type:
 - For John Mathews → **John**
 - For Priya Suresh → **Priya**
4. Power Query auto-generates M-code:

```
Text.BeforeDelimiter([Full Name], " ")
```

Output

Full Name	First Name
John Mathews	John
Priya Suresh	Priya
Rohan Kumar	Rohan

2. Extract Domain From Email

Business Requirement:

“Extract the domain name from the email.”

Email: john.m@abc.com

Domain needed: **abc.com**

Steps:

Add Column → Column From Examples → From Selection

Type examples:

- For john.m@abc.com → **abc.com**
- For priya.suresh@xyz.in → **xyz.in**

Power Query auto-generates:

`Text.AfterDelimiter([Email], "@")`

Output

Email	Domain
john.m@abc.com	abc.com
priya.suresh@xyz.in	xyz.in

rohan.kumar@mega.co.in	mega.co.in
------------------------	------------

3. Format Phone Numbers

Current:

9876543210

Required format:

987-654-3210 (just an example)

Business Requirement:

“Convert phone numbers into formatted pattern.”

Steps:

Select **Phone** → Column From Examples.

Type:

- For 9876543210 → **987-654-3210**

Power Query automatically identifies pattern and generates formula like:

`Text.Middle([Phone], 0, 3) & "-" & Text.Middle([Phone], 3, 3) & "-" & Text.Middle([Phone], 6, 4)`

Output

Phone	Formatted Phone
9876543210	987-654-3210
9567812345	956-781-2345
9123456789	912-345-6789

4. Auto Detect Patterns

Power Query automatically picks patterns when:

- Extracting first/last names
 - Extracting ID from codes (e.g., ORD-9852 → 9852)
 - Splitting based on spaces/dots/dashes
 - Creating codes like EMP-101, EMP-102
 - Concatenating multiple fields
-

Example Dataset

Code
EMP-001
EMP-002
EMP-003

Business Requirement:

“Extract only the numeric part.”

Steps:

Type:

- 001
- 002

Power Query auto-generates:

Text.AfterDelimiter([Code], "-")

DATA TYPE CONVERSION

Every column in Power Query must have a correct **data type**, because:

- ✓ Calculations depend on it
- ✓ Sorting & filtering change
- ✓ Date functions work only on date types
- ✓ Text functions work only on text types

Supported Data Types

Data Type	Used For	Example Values
Whole Number	Counts, ID fields	10, 250, 1001
Decimal Number	Amounts, percentages, averages	1250.50
Text	Names, emails, codes	"John", "ORD-123"
Date	Order date, join date	2024-03-05
Date/Time	Timestamps	2024-03-05 12:45
Boolean	TRUE/FALSE conditions	TRUE, FALSE

Sample Dataset

OrderID	Order Date	Amount	Customer	Active
101	05-03-2024	1500.75	John	TRUE
102	12-03-2024	900	Priya	FALSE
103	2024/03/18	450.5	Kiran	TRUE
104	18-03-2024	1750	Anu	TRUE

1. Whole Number Conversion

Used when the column contains **IDs, counts, quantity**.

Example:

- OrderID = 101 → Whole Number
- Quantity = 5 → Whole Number

Steps

Home → Data Type → Whole Number

Problem if incorrect

If text values like “001” exist → converting to number becomes **1**
(leading zeros lost)

2. Decimal Number Conversion

Used for:

- ✓ Amounts
- ✓ Prices
- ✓ Discounts
- ✓ Percentages

Example:

- 1500.75 → Decimal Number
- 900 → Decimal Number

Power Query automatically converts integers (900) to 900.00 (decimal type).

3. Text Conversion

Used for:

- ✓ Names
- ✓ Product Codes
- ✓ Phone numbers
- ✓ Customer IDs (with letters)

Example Problem:

If phone number 9876543210 becomes a **Whole Number**, it may show as **9.87654E+09** (scientific format).

Solution → Convert to **Text**.

4. Date Conversion

Used for:

- ✓ Order Date
- ✓ Invoice Date
- ✓ Join Date

Power Query supports many date formats:

- 05-03-2024
 - 2024/03/18
 - 03/05/2024
 - 2024-03-05
-

After conversion, Power Query shows a calendar icon

- ✓ Date recognized
 - ✗ If error appears → wrong format
-

5. Date/Time Conversion

Used when column contains timestamp:

Example:

Date/Time
2024-03-05 10:45 AM
2024-03-06 16:32

Used for:

- ✓ Log files
 - ✓ Tracking updates
 - ✓ Reporting refresh times
-

6. Boolean Conversion (TRUE/FALSE)

Used for logical fields:

- Active
- Passed/Failed
- Available/Not available

Power Query accepts:

Text	Converted Boolean
"TRUE"	TRUE
"FALSE"	FALSE
"Yes/No"	(must clean first)

Common Data Type Issues & Fixes

1. Wrong Date Format

Example:

- "05-03-2024" interpreted as **May 3rd** instead of **March 5th**

Fix:

Use *Locale Conversion*

Transform → Data Type → **Using Locale**

Choose

- Data Type = Date
 - Locale = English (India) or dd-MM-yyyy
-

2. Text interpreted as Number

Examples:

Value	Problem
00123	Converted to 123 (leading zeros lost)
9876543210	Shown as 9.87654E+09

Fix → Set Data Type = **Text**

3. Mixed Data Types

Example:

Value
1500
1800.50
“N/A”

Converting to Decimal → errors appear.

Fix sequence:

1. Replace “N/A” → null
 2. Change type → Decimal Number
-

4. Date column containing time

If time exists but type is only **Date** → time is removed.

Fix → Set **Date/Time** type

PIVOT & UNPIVOT in Power Query

These are two of the MOST powerful data transformation features.

1. PIVOT COLUMNS

Pivot = Convert rows → columns

Used to *summarize* data by spreading values across new columns.

Real Business Example (Before Pivot)

Sales Table (Unpivoted)

Product	Year	Sales
Laptop	2022	150000
Laptop	2023	180000
Tablet	2022	90000
Tablet	2023	120000
Mobile	2022	220000
Mobile	2023	250000

Goal:

Convert data to this format:

Product	2022 Sales	2023 Sales
Laptop	150000	180000
Tablet	90000	120000
Mobile	220000	250000

How to Pivot in Power Query

Steps:

1. Select: **Year column** → this becomes new column headers
 2. Select **Transform** → **Pivot Column**
 3. Values Column = **Sales**
 4. Aggregation = **Sum** (default)
-

Why Pivot is useful?

- ✓ Compare year-wise sales
 - ✓ Create reporting format
 - ✓ Easy to visualize in Power BI visuals
 - ✓ Convert long (tall) data into wide layout
-

More Pivot Examples

Example: Region-wise Sales (Pivot Region)

Before:

Region	Month	Sales
South	Jan	25000
North	Jan	30000
South	Feb	27000
North	Feb	31000

After Pivot (Region as columns):

Month	South	North
Jan	25000	30000

Feb	27000	31000
-----	-------	-------

2. UNPIVOT COLUMNS

Unpivot = Convert columns → rows

Opposite of pivot. Makes data *longer*, ideal for analysis.

Real Business Example (Before Unpivot)

Sales Report (wide format)

Product	Jan	Feb	Mar
Laptop	50000	52000	51000
Mobile	70000	68000	69000
Tablet	30000	32000	31000

Goal: Convert Jan, Feb, Mar columns into rows:

Product	Month	Sales
Laptop	Jan	50000
Laptop	Feb	52000
Laptop	Mar	51000
Mobile	Jan	70000
Mobile	Feb	68000
Mobile	Mar	69000
Tablet	Jan	30000

Tablet	Feb	32000
Tablet	Mar	31000

How to Unpivot in Power Query

Steps:

1. Select column(s) to **keep** (Product)
2. Right-click → **Unpivot Other Columns**

Power Query creates:

- **Attribute** column → Month
- **Value** column → Sales

Why Unpivot is VERY important?

- ✓ Converts Excel-like reports into usable data
- ✓ Required for date intelligence
- ✓ Helps DAX work correctly
- ✓ Best for visuals (e.g., Month axis needs rows)

More Unpivot Examples

Example: Employee Attendance (Wide → Long)

Before:

Emp	Jan	Feb	Mar
John	P	P	A
Priya	A	P	P

After Unpivot:

Emp	Month	Status
John	Jan	P
John	Feb	P
John	Mar	A
Priya	Jan	A
Priya	Feb	P
Priya	Mar	P

FORMATTING COLUMNS

Formatting columns is used to **clean text**, **standardize values**, and **fix numeric formats** before loading data into Power BI.

Sample Dataset

CustomerID	Customer Name	Email	Sales Amount	Notes
C001	john doe	JOHN.DOE@GMAIL.COM	2500.5678	Extra spaces
C002	priya kumar	priya.kumar@yahoo.com	999.4	Contains tab
C003	Rohan-SHETTY	ROHAN.SHETTY@OUTLOOK.COM	1200.999	Bad characters

A. TEXT FORMATTING

Trim

Removes **leading and trailing spaces**.

Before → After

Customer Name (Raw)	Trimmed
" john doe "	"john doe"
"priya kumar"	"priya kumar"

Removes **ONLY** spaces at start/end

Does **NOT** remove extra spaces in middle.

Clean

Removes **non-printable characters** (tabs, ASCII control chars).

Before → After

Notes (Raw)	Cleaned
"Extra spaces "	"Extra spaces"
"Contains\t tab"	"Contains tab"
"Bad characters ☺"	"Bad characters ☺" <i>(since ☺ is printable, clean does not remove)</i>

UPPER

Converts text → **ALL CAPITAL letters**.

Email (Raw)	UPPER()
john.doe@gmail.com	JOHN.DOE@GMAIL.COM
priya.kumar@yahoo.com	PRIYA.KUMAR@YAHOO.COM

Use Case: Standardize email text before matching.

lower

Converts text → **all lowercase letters**.

Email (Raw)	lower()
JOHN.DOE@GMAIL.COM	john.doe@gmail.com

Use Case: Email comparison, Merge operations.

Proper Case

Converts text → **First letter capital** of each word.

Customer Name (Raw)	Proper Case
"john doe"	John Doe

"priya kumar"	Priya Kumar
"ROHAN-SHETTY"	Rohan-Shetty

Use Case: Display-friendly customer names.

Text Formatting Summary

Raw Value	Trim	Clean	UPPER	lower	Proper Case
" john doe "	john doe	john doe	JOHN DOE	john doe	John Doe

B. NUMBER FORMATTING

Sample numbers used:

2500.5678

999.4

1200.999

Round

Round to nearest number.

Value	Round(2)	Explanation
2500.5678	2500.57	Rounds to 2 decimals
999.4	999.40	Adds decimal padding
1200.999	1201.00	Rounds up due to 3rd digit

Round Up

Always rounds **up to next number**.

Value	Round Up(1)	Explanation
-------	-------------	-------------

2500.5678	2500.6	goes up
1200.999	1201.0	goes up

Round Down

Always rounds **down** (floor).

Value Round Down(1)

2500.5678 2500.5

1200.999 1200.9

Divide / Add new calculated numeric fields

Example 1 — Convert sales to thousands

Formula:

[Sales Amount] / 1000

Sales Amount	Sales in K
2500.5678	2.5

Example 2 — Add 18% GST

Formula:

[Sales Amount] * 1.18

Sales Amount	Price w/ GST
2500.5678	2950.67

Example 3 — Discounted Price

[Sales Amount] * (1 - 0.10) (10% discount)

Sales Amount	After Discount
1200.999	1080.90

Business Use Cases

Standardizing Customer Names

Proper case ensures names look professional in reports.

Cleaning Emails

lower() ensures consistent matching during Merge / Join.

Improving Numeric Accuracy

Rounding ensures KPI values match financial requirements.

Removing Junk Characters

Clean() ensures Excel/CSV imported data has no weird symbols.

DATE FUNCTIONS

Sample Dataset

OrderID	Order Date	Delivery Date	Customer DOB
O001	2024-02-15	2024-02-20	1995-06-10
O002	2023-11-28	2023-12-05	1988-04-02
O003	2024-01-05	2024-01-08	2000-10-25

PART 1: Create New Date Columns

Year

Extracts **year number**.

Order Date	Year
2024-02-15	2024
2023-11-28	2023

Power Query Transform Menu:
Add Column → Date → Year → Year

Month (Number)

Extracts **month number (1–12)**.

Order Date	Month
2024-02-15	2
2023-11-28	11

Month Name

Full month name.

Order Date	Month Name
2024-02-15	February
2023-11-28	November

Options:

- "Name of Month"
- "First 3 letters" (Feb, Nov)

Quarter

Extracts quarter number Q1–Q4.

Order Date	Quarter
2024-02-15	Q1
2023-11-28	Q4

Week Number

Returns week of the year (1–52).

Order Date	Week No.
2024-02-15	7
2023-11-28	48

Used in:

Sales forecasting

Weekly performance dashboards

Day Name

Full day name (Monday, Tuesday).

Order Date	Day Name
2024-02-15	Thursday
2023-11-28	Tuesday

Also available:

- First letter ("T")
- Short name ("Thu")

PART 2: Duration Functions

Duration functions calculate the **difference between two dates** or the **age** of a customer.

Date Difference (Delivery Days)

Formula:

[Delivery Date] - [Order Date]

Output

Order Date	Delivery Date	Difference (days)
2024-02-15	2024-02-20	5 days
2023-11-28	2023-12-05	7 days
2024-01-05	2024-01-08	3 days

Business use:

Delivery time KPI

SLA comparison

Shipping performance

Age (Based on DOB)

Power Query →

Add Column → Date → Age

This returns a duration in days, hours, seconds.

Then convert to **Years** by:

Transform → Duration → Total Years

Example:

Customer DOB	Age (Years)
1995-06-10	29.6
1988-04-02	36.7
2000-10-25	24.2

Used in:

- ✓ Age banding
 - ✓ Customer segmentation
-

Duration in Days, Hours, Minutes

Duration = Delivery Date – Order Date

For O001:

2024-02-20 – 2024-02-15 = **5 days**

Power Query provides:

- **Total Days**
- **Total Hours**
- **Total Minutes**
- **Total Seconds**

Example Output:

OrderID	Duration	Total Days	Total Hours	Total Minutes
O001	5.00:00:00	5	120	7200
O002	7.00:00:00	7	168	10080

Complete Summary Table

Function	Input Columns	Output Example	Usage
Year	Order Date	2024	Sales by Year
Month	Order Date	2	Seasonal analysis
Month Name	Order Date	February	Reports
Quarter	Order Date	Q1	Q1/Q2/Q3 dashboards
Week Number	Order Date	7	Weekly sales
Day Name	Order Date	Thursday	Customer footfall pattern
Date Difference	Order-Delivery	5 days	Delivery KPI
Age	DOB	29.6 years	Segmentation
Duration in Days/Hours	Date pairs	120 hrs	SLA calculations

POWER QUERY (PARAMETERS)

Power Query **Parameters** allow you to make your queries **dynamic**.

You can use parameters to control:

- Folder path

- Date ranges
 - Filtering rules
 - File names
 - Threshold values
-

Types of Parameters

Power Query supports **5 main parameter types**:

1. **Text**
2. **Number**
3. **List of Values**
4. **Query-Based List**
5. **Current Value**

We will explain each with examples.

Example Dataset (for filtering)

OrderID	Order Date	Sales	Region
O001	2024-01-18	1200	South
O002	2024-02-02	1800	East
O003	2024-02-10	900	South
O004	2024-03-01	1500	West
O005	2024-04-15	2000	North

1. TEXT PARAMETER

Used for text-based filtering or folder/file paths.

Example 1: Filter Sales by Region (Dynamic)

Create a parameter:

Name: RegionParam

Type: Text

Current Value: South

Now apply it:

Power Query → **Filter Rows** → **Equals** → **RegionParam**

Output after applying parameter

OrderID	Order Date	Sales	Region
O001	2024-01-18	1200	South
O003	2024-02-10	900	South

Example 2: Folder Path Parameter

Name: FolderPath

Type: Text

Value: C:\Company\Data\Sales2024

Used with:

Home → Get Data → Folder → use parameter instead of typing path.

Business Use:

- ✓ Allows user to change folder without editing query
- ✓ Used in monthly file automation

2. NUMBER PARAMETER

Used for numeric thresholds, limits, minimum values, etc.

Example: Filter Sales Greater Than a Dynamic Value

Create:

Name: MinSales

Type: Number

Value: 1500

Apply:

Filter rows → Sales > MinSales

✓ Output

OrderID	Sales
O002	1800
O004	1500
O005	2000

Only values > 1500 appear.

3. LIST OF VALUES PARAMETER

You create a parameter with **multiple allowed values**.

Example allowed values:

South

North

East

West

Example: Choose Region at Refresh Time

Parameter:

- Name: RegionList
- Type: **List of Values**
- Allowed values: South, East, West
- Current Value: East

Apply filter → Region = RegionList

✓ Output

OrderID	Region	Sales
O002	East	1800

This is useful for:

- ✓ User dropdown
 - ✓ Multiple region selection
 - ✓ Dynamic slice-and-refresh reports
-

4. QUERY-BASED LIST

Parameter values come from another query.

Example Data: List of Actual Regions

Create a query RegionMaster:

Region

North

South

East

West

Now create a parameter:

- Type: **Query**
- Query: RegionMaster
- Current Value: South

This allows dynamic values that come from real data.

Business Use:

- ✓ When region list changes, parameter list updates
 - ✓ Prevents manually typing values
-

5. CURRENT VALUE

This is the value *Power Query will actually use* during transformations.

Every parameter has:

- **Allowed values**
- **Current value**

Example:

Parameter: MonthParam

Values: 1 to 12

Current value: 2

Meaning: Only February data is loaded.

Real-Time Use Cases of Parameters

Dynamic Filtering (Text + Number Parameters)

Example:

Filter Order Date $\geq$ StartDate parameter.

Parameter:

StartDate = 2024-02-01

Filter:

Order Date $\geq$ StartDate

✓ Output

OrderID	Order Date
O003	2024-02-10
O004	2024-03-01

O005	2024-04-15
------	------------

Dynamic Folder Path

Most common in companies.

Parameter:

FolderPath = C:\Sales\2024\Feb

Power Query:

Folder.Files(FolderPath)

Used for:

- ✓ Monthly file loading
 - ✓ Changing folders without editing M code
-

Dynamic Start Date / End Date

Create:

- StartDate (2024-01-01)
- EndDate (2024-03-01)

Filter:

OrderDate >= StartDate

AND

OrderDate <= EndDate

✓ Output

OrderID	Order Date
O001	2024-01-18
O002	2024-02-02

O003	2024-02-10
------	------------

FULL SUMMARY TABLE

Parameter Type	Example	Business Use
Text	RegionParam = "South"	Filter region dynamically
Number	MinSales = 1500	Dynamic KPI threshold
List of Values	East, West, South	User selection dropdown
Query-Based	Values from RegionMaster	Auto-updating list
Current Value	Active region / month	Real-time filtering

MANAGE QUERIES IN POWER QUERY

We have a sample table:

Sales_Data

OrderID	Customer	Product	Amount	OrderDate
101	Ram	Laptop	55000	2024-01-05
102	John	Mouse	800	2024-01-06
103	Priya	Keyboard	1200	2024-01-06
104	Arjun	Laptop	60000	2024-01-07

1. Duplicate Query

What it does:

Creates a **full independent copy** of the query.

✓ When to use

- If you want to create a **completely separate version** for different transformations
- Original query and duplicate are **not connected**

✓ Example

You duplicate **Sales_Data** → **Sales_Analytics**

Now you can add:

- Calculated columns
 - Additional cleaning
 - Remove columns
Without affecting the original.
-

2. Reference Query

What it does:

Creates a **new query that depends on the original**.

If the original changes → Referenced query also updates.

✓ When to use

- To create **Staging + Final layer**
- To reuse cleaned data
- To avoid repeating the same cleaning steps

✓ Example

You create **Sales_Staging** → Apply cleaning

Then **Reference** it to create:

- **Sales_Report_View** (for reporting)
- **Sales_Finance_View** (for finance-specific fields)

If you fix something in **Sales_Staging**, both views get updated automatically.

3. Rename Query

Simple but important.

✓ When to use

- When defining layers (Raw / Staging / Final)
- When preparing data models
- For clarity in large projects

Example naming structure:

- **Raw_Sales**
 - **Staging_Sales**
 - **Final_Sales**
-

4. Organize Queries into Folders (Groups)

Used for better project structure.

✓ Example Groups

- **RAW DATA**
- **STAGING**
- **FINAL MODELS**

- **PARAMETERS**
- **FUNCTIONS**

Why useful?

Easier to manage **20+ queries** in a project.

When to Use REFERENCE – Practical Example

Sample workflow

Step 1 — Import Raw Data

Query name: **Sales_Raw**

OrderID	Customer	Amount	OrderDate
101	Ram	55000	01/05/24
102	John	800	06-01-2024
103	Priya	1200	6 Jan 24

Problems:

- Date formats differ
 - Customer names inconsistent
 - Amount may have text values
-

Step 2 — Create Staging Layer

Right-click → **Reference** → **Sales_Staging**

Apply cleaning:

- ✓ Fix date column
- ✓ Trim customer names
- ✓ Change data types
- ✓ Remove duplicates

Now **Sales_Staging** is the **CLEAN** base layer.

Step 3 — Create Final Layer

From **Sales_Staging** → Right-click → **Reference**

Create:

Final_Sales_Report

Add:

- Year column
- Month name
- Revenue category
- Final selected columns

Final_Sales_Finance

Add:

- Tax column
- Profit calculation
- Only finance-required columns

WHY THIS MATTERS

Without Reference (Only Duplicate)

- If source changes → Duplicate will NOT update
- You may need to repeat cleaning steps for each query → More work

With Reference

- Clean once in Staging
- Reuse the cleaned version for all final outputs
- Fixing 1 issue updates all dependent queries

This is how **real-time BI teams structure Power BI ETL flows**.

Summary Chart

Feature	Purpose	Connected to Original?	Real Use Case
Duplicate	Copy query	No	Create separate analytics copy
Reference	Create dependent query	Yes	Staging → Final model
Rename	Give meaningful names	—	Better readability
Groups	Organize queries	—	Clean project structure

Query Dependencies

When you click **View → Query Dependencies**, Power BI displays a *visual map* of how your queries are connected.

It shows:

✓ 1. Data Flow

How data moves from **source → transformations → final tables**.

✓ 2. Parent / Child Queries

- **Parent** = query that another query *depends on*.
- **Child** = query that *references* or uses the output of another query.

✓ 3. Staging → Transformation → Final Output Layers

This is the **recommended Power BI ETL structure**.

Sample Example

Let's use three simple tables:

Sales_Raw (Source Data)

OrderID	Customer	Amount	OrderDate
101	Ram	55000	01/05/24
102	John	800	06/01/24
103	Priya	1200	06-01-24

STEP 1 — Create STAGING Query

Right-click **Sales_Raw → Reference → Sales_Staging**

Apply cleaning:

- ✓ Fix date formats
- ✓ Trim customer names

✓ Remove duplicates

✓ Correct data types

Sales_Staging Output

OrderID	Customer	Amount	OrderDate
101	Ram	55000	2024-01-05
102	John	800	2024-01-06
103	Priya	1200	2024-01-06

STEP 2 — Create TRANSFORMATION Query

Right-click **Sales_Staging** → **Reference** → **Sales_Transform**

Add calculated columns:

✓ Year

✓ Month Name

✓ Revenue Category (High / Medium / Low)

Sales_Transform Output

OrderID	Amount	Year	Month	Category
101	55000	2024	January	High
102	800	2024	January	Low
103	1200	2024	January	Low

STEP 3 — Create FINAL Query

Right-click **Sales_Transform** → **Reference** → **Sales_Final**

Keep only columns needed for report visuals:

- ✓ Year
- ✓ Month
- ✓ Total Amount

Sales_Final Output

Year	Month	Total Sales
2024	January	57000

How Query Dependencies Will Look

In Query Dependency View, you will see the following structure:

Sales_Raw

↓

Sales_Staging

↓

Sales_Transform

↓

Sales_Final

Interpreting the Dependency Diagram

✓ Sales_Raw

Parent for **Sales_Staging**

✓ Sales_Staging

Parent for **Sales_Transform**

✓ Sales_Transform

Parent for **Sales_Final**

✓ Sales_Final

Final output used in reports
(No child queries)

What Query Dependencies Helps With

1. Understand Complete Data Flow

Shows how data travels from source → report-ready table.

2. Identify Impact of Changes

If you edit **Sales_Staging**,
→ Sales_Transform and Sales_Final will change automatically.

3. Avoid Breaking the Model

You can see which queries depend on each other.

4. Perfect for Enterprise BI Workflow

Helps enforce:

- RAW layer
 - STAGING layer
 - TRANSFORMATION layer
 - FINAL layer
-

Real-Time Project Example (Short)

A corporate BI team might structure queries like:

Excel Source → Raw_Sales → Staging_Sales → Sales_Cleaned → Final_Sales

SQL Database → Raw_Customer → Staging_Customer → Dim_Customer

ADVANCED EDITOR (M-CODE)

The **Advanced Editor** shows the **behind-the-scenes code** that Power Query generates for all transformations.

You can open it from:

Home → Advanced Editor

Sample Data for This Topic

Let's use a simple Sales table:

Sales_Data

OrderID	Customer	Amount	OrderDate
101	Ram	55000	2024-01-05
102	John	800	2024-02-11
103	Priya	1200	2024-02-16
104	Ram	3000	2024-03-02

1. Understanding M-Language Structure

Every M-query follows this structure:

let

 StepName1 = ...,

 StepName2 = ...,

 StepName3 = ...

in

 StepName3

✓ let

Defines all steps

✓ in

Outputs the final result

✓ Steps are applied in order (top to bottom)

2. Viewing Auto-Generated M Code

If you load the above sample data (Excel/CSV), Power Query generates code:

let

```
Source = Excel.Workbook(File.Contents("C:\Files\Sales.xlsx"), null, true),
Data_Sheet = Source[[Item="Sales",Kind="Sheet"]][Data],
PromotedHeaders = Table.PromoteHeaders(Data_Sheet, [PromoteAllScalars=true]),
ChangedTypes = Table.TransformColumnTypes(PromotedHeaders,
    {"OrderID", Int64.Type}, {"Customer", type text},
    {"Amount", Int64.Type}, {"OrderDate", type date})
```

in

```
ChangedTypes
```

Students can clearly see:

- ✓ Import
 - ✓ Promote headers
 - ✓ Change types
-

3. Editing Steps in Advanced Editor

Business Requirement

Add a calculated column:

Category = "High" if Amount > 10,000 else "Low"

Solution: Edit M-Code

Add new step below ChangedTypes:

```
AddedCategory = Table.AddColumn(ChangedTypes, "Category",
    each if [Amount] > 10000 then "High" else "Low")
```

Full Query:

let

```
Source = Excel.Workbook(File.Contents("C:\Files\Sales.xlsx"), null, true),
```

```

Data_Sheet = Source([Item="Sales",Kind="Sheet"])[Data],
PromotedHeaders = Table.PromoteHeaders(Data_Sheet, [PromoteAllScalars=true]),
ChangedTypes = Table.TransformColumnTypes(PromotedHeaders,
    {{"OrderID", Int64.Type}, {"Customer", type text},
    {"Amount", Int64.Type}, {"OrderDate", type date}}),

AddedCategory = Table.AddColumn(ChangedTypes, "Category",
    each if [Amount] > 10000 then "High" else "Low")
in
    AddedCategory

```

What students learn

- How steps are added
- How to modify logic
- Difference between UI steps vs code steps

4. Writing Custom Logic (Manually)

Business Requirement

Create a Discount column:

- If Amount > 20,000 → 10%
- Else → 2%

Add this step:

```

AddedDiscount = Table.AddColumn(AddedCategory, "Discount",
    each if [Amount] > 20000 then 0.10 else 0.02)

```

Now full pipeline ends with:

```

in
    AddedDiscount

```

Students understand:

- Nested IF
- Adding a new step
- Referencing previous step names

5. Combining Multiple Transformations in M-Code

Requirement

Convert Customer to UPPER

Add Year column

Sort by Amount

Add these steps:

```
UpperCustomer = Table.TransformColumns(AddedDiscount,  
    {"Customer", Text.Upper})),
```

```
AddedYear = Table.AddColumn(UpperCustomer, "Year",  
    each Date.Year([OrderDate]), Int64.Type),
```

```
Sorted = Table.Sort(AddedYear, {"Amount", Order.Ascending}))
```

Full output:

in

Sorted

Final Output After All Transformations

OrderID	Customer	Amount	OrderDate	Category	Discount	Year
---------	----------	--------	-----------	----------	----------	------

102	JOHN	800	2024-02-11	Low	0.02	2024
103	PRIYA	1200	2024-02-16	Low	0.02	2024
104	RAM	3000	2024-03-02	Low	0.02	2024
101	RAM	55000	2024-01-05	High	0.10	2024

What Students Learn Using This Example

- ✓ How M-code is structured
 - ✓ How every UI transformation becomes code
 - ✓ How to write conditions manually
 - ✓ How to combine multiple transformations
 - ✓ How to create reusable transformations
 - ✓ Why M-code gives more power than UI
-

ERROR HANDLING IN POWER QUERY

Error handling is critical in Data Analytics because dirty/incorrect data can break the entire model.

Power Query provides multiple tools:

- ✓ **Remove Errors**
- ✓ **Replace Errors**
- ✓ **Column Quality**
- ✓ **Column Distribution**
- ✓ **Column Profile**

We will explain each with **sample dataset + business scenarios**.

Sample Data

Sales_Data

OrderID	Customer	Amount	OrderDate
101	Ram	5000	2024-01-04
102	Priya	ERROR	2024-02-10
103	John	3000	invalid-date
104	Meera	7000	2024-03-05
NA	NULL	9000	2024-04-01
105	Ram	1500	2024-02-20

Error Types in this dataset:

- Amount: **Error value** (non-numeric)
- OrderDate: **Invalid date**
- OrderID: **NULL**
- Customer: **NULL**

1. Column Quality – First Step to Detect Issues

Turn on:

✓ **View → Data Preview → Column Quality**

It shows % **Valid** | % **Error** | % **Empty**

For our sample data:

Column	Valid	Error	Empty
OrderID	83%	0%	17%
Customer	83%	0%	17%
Amount	83%	17%	0%

OrderDate	83%	17%	0%
-----------	-----	-----	----

✓ What students learn:

This helps identify **which columns have errors before applying transformations.**

2. Column Distribution

Turn on:

✓ View → Column Distribution

It shows:

- Unique values
- Value frequency
- Data pattern

Example:

Customer column distribution

- Ram: 2
- Priya: 1
- John: 1
- NULL: 1

✓ Helps detect anomalies (e.g., too many NULLs).

3. Column Profile

Turn on:

✓ View → Column Profile

Shows advanced details:

- Data type
- Max/Min
- Average
- Error count
- Distinct count

- Empty count

Example:

Amount column profile

- Min: 1500
- Max: 9000
- Errors: 1
- Nulls: 0

✓ Allows deep validation of column data before cleaning.

4. Remove Errors

Used when:

- ✓ Error rows are NOT required
- ✓ Data is large & errors are rare
- ✓ Error rows are invalid / corrupt

Example Problem

Amount column has an error for Order 102.

Steps

Home → Remove Errors

Result

OrderID	Customer	Amount	OrderDate
101	Ram	5000	2024-01-04
103	John	3000	invalid-date
104	Meera	7000	2024-03-05
NA	NULL	9000	2024-04-01
105	Ram	1500	2024-02-20

→ Only the row with "ERROR" in Amount is removed

✓ Use when **bad rows should not be kept at all.**

5. Replace Errors

Used when:

- ✓ You want to keep the record
- ✓ Only fix the error value

Example Problem

Amount for Order 102 shows an error.

Use:

Transform → Replace Errors

Enter replacement value:

0 or "Unknown" or average amount

Result:

OrderID	Customer	Amount	OrderDate
102	Priya	0	2024-02-10

✓ Replaces **ONLY** the error value, not the whole row.

Equivalent M-Code

= Table.ReplaceErrorValues("#Previous Step", {"Amount", 0})

6. Fix Date Errors Using Replace Errors

Problem

OrderDate has “invalid-date”.

Instead of removing the entire row:

Replace error → Enter default date

Example: #date(2024, 01, 01)

Result

OrderID	Customer	OrderDate
103	John	2024-01-01

✓ Good for converting dirty dates into a standard format.

7. Fix Missing Data (NULL Values)

Use "Replace Values":

Replace NULL Customer → “Unknown”

OrderID	Customer
NA	Unknown

Students learn how to treat missing values properly.

8. Business Use Cases (Why Error Handling Matters)

Sales Department

Incorrect amount → Wrong revenue → Wrong dashboard

Finance

Invalid dates → Incorrect aging reports

HR

Null employee ID → Incorrect headcount reports

Supply Chain

Error in quantity → Wrong stock analysis

Final Clean Dataset (After Fixes)

OrderID	Customer	Amount	OrderDate
101	Ram	5000	2024-01-04

102	Priya	0	2024-02-10
103	John	3000	2024-01-01
104	Meera	7000	2024-03-05
NA	Unknown	9000	2024-04-01
105	Ram	1500	2024-02-20

What Students Learn After This Lesson

- ✓ Detecting errors
 - ✓ Fixing error values
 - ✓ Removing corrupted rows
 - ✓ Understanding data quality
 - ✓ Using advanced profiling tools
 - ✓ How error handling affects reports
 - ✓ When to remove vs when to fix errors
 - ✓ Real-world problem-solving
-